

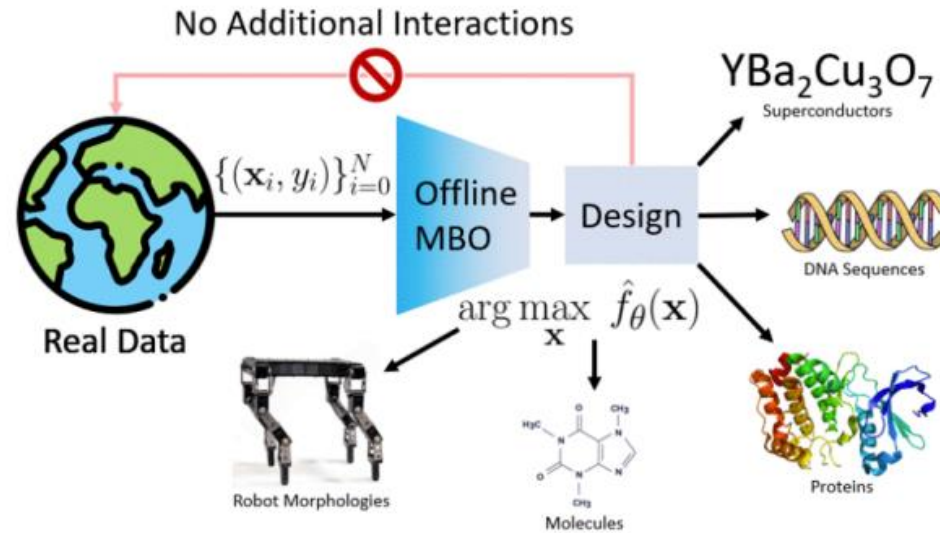
IE437 Data-Driven Decision Making and Control
2024 Spring Semester

5. Data-Driven Design Optimization (Surrogate Model Based)

Jinkyoo Park
Associate Professor
Industrial and Systems Engineering Department

Recent Papers

Motivation



- In real world optimization problems, evaluation of objective function is **expensive** and **prohibitive** (Examples) Molecule Design, Automated Aircraft Design, ...
- Therefore, constructing a data-driven model given a fixed, offline dataset is crucial
- Formally, we try to find an input $\mathbf{x}$ such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmax}} f(\mathbf{x}) \text{ with **only** a fixed dataset, } D = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_N, f(\mathbf{x}_N))\}$$

Motivation

- Formally, we try to find an input $\mathbf{x}$ such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmax}} f(\mathbf{x}) \text{ with } \mathbf{only} \text{ a fixed dataset, } D = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_N, f(\mathbf{x}_N))\}$$

- Main Approaches
 - Surrogate model-based:** Approximate $f(x)$ using previously collected data, choose the query that maximizes the surrogate model
 - Generative model-based:** Learn an inverse function $f^{-1}(y)$ using previously collected data, choose the query generated from the inverse function given by a desired output.
- Today, we'll focus on Surrogate model-based approaches

Prior Works – Naïve Approach

- A straightforward way to solve Design Optimization is by estimating a ground-truth function with a feed-forward model using a supervised learning framework and performing gradient ascent to find the optimal point

$$\textbf{Step 1. } \theta^* = \operatorname{argmin}_{\theta} \frac{1}{N} \sum_{i=1}^N (f_{\theta}(\mathbf{x}_i) - f(\mathbf{x}_i))^2 \text{ where } D = \{(\mathbf{x}_i, f(\mathbf{x}_i))\}_{i=1}^N$$

$$\textbf{Step 2. choose } \mathbf{x}^* = \operatorname{argmax}_{\mathbf{x}} f_{\theta^*}(\mathbf{x})$$

Prior Works – Naïve Approach

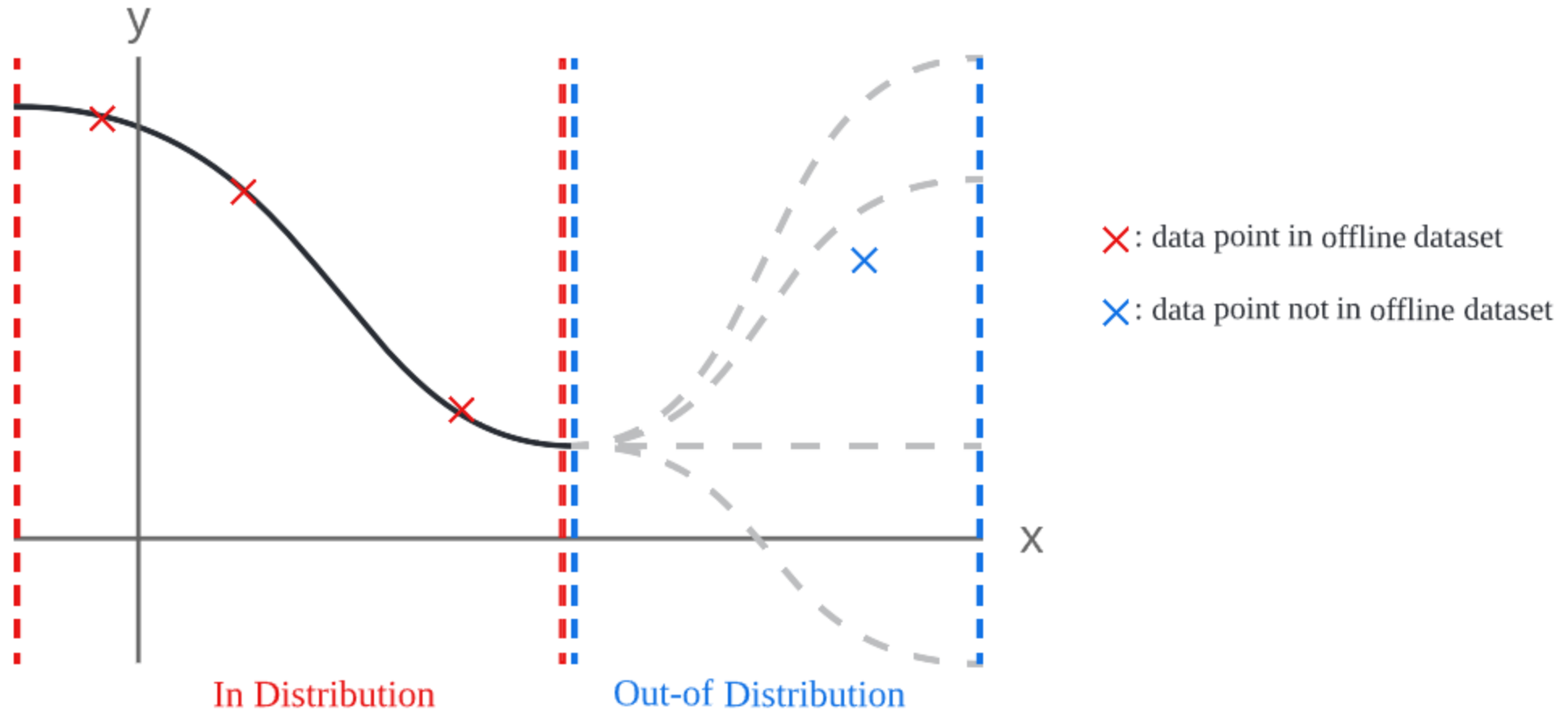
- A straightforward way to solve Design Optimization is by estimating a ground-truth function with a feed-forward model using a supervised learning framework and performing gradient ascent to find the optimal point

$$\text{Step 1. } \theta^* = \operatorname{argmin}_{\theta} \frac{1}{N} \sum_{i=1}^N (f_{\theta}(\mathbf{x}_i) - f(\mathbf{x}_i))^2 \text{ where } D = \{(\mathbf{x}_i, f(\mathbf{x}_i))\}_{i=1}^N$$

$$\text{Step 2. choose } \mathbf{x}^* = \operatorname{argmax}_{\mathbf{x}} f_{\theta^*}(\mathbf{x})$$

Problem 1. The trained model is only valid near the training distribution, which leads to a large error in out-of-distribution input values. However, we must extrapolate from the training distribution!!

Prior Works – Naïve Approach



Prior Works – Naïve Approach

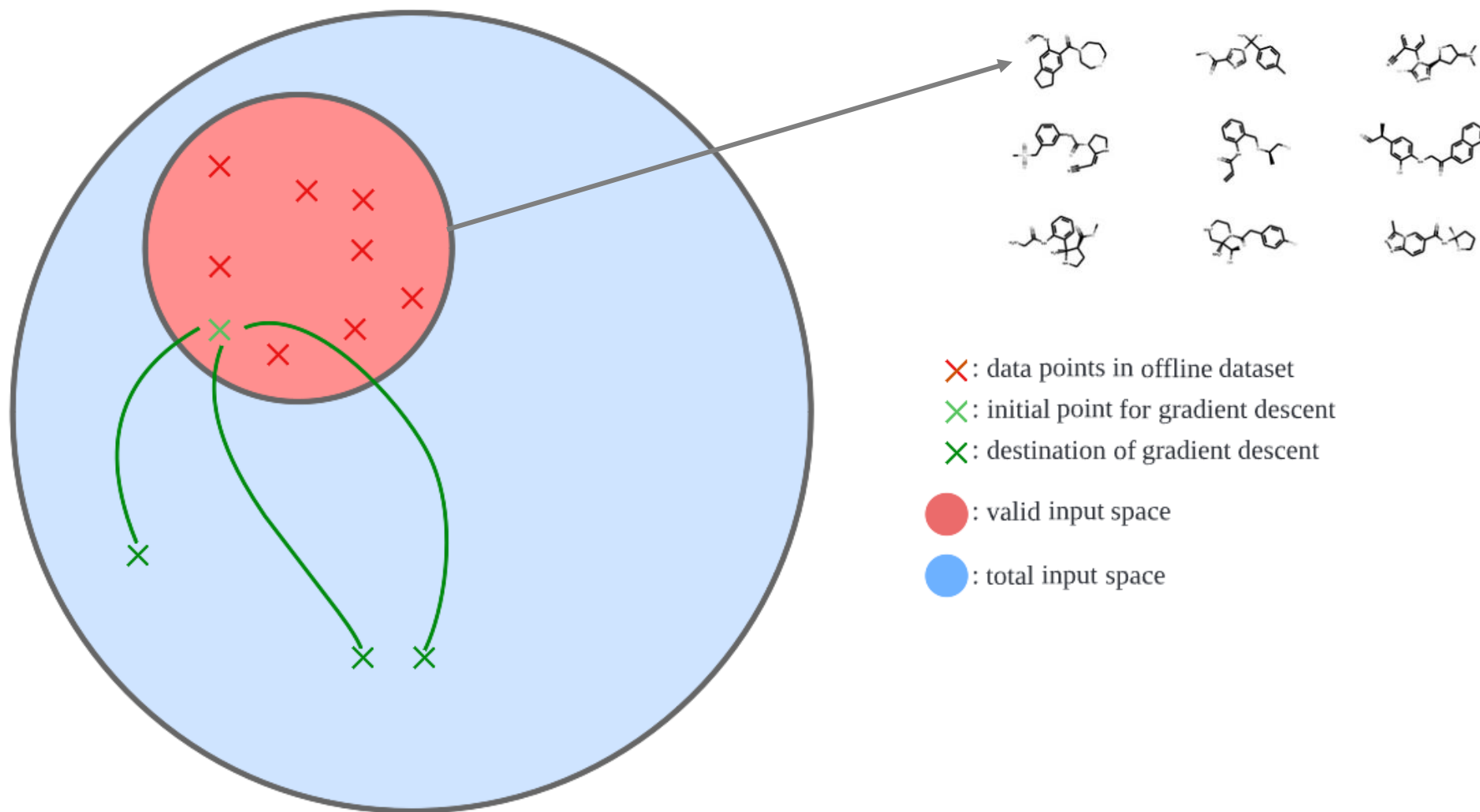
- A straightforward way to solve Design Optimization is by estimating a ground-truth function with a feed-forward model using a supervised learning framework and performing gradient ascent to find the optimal point

$$\text{Step 1. } \theta^* = \operatorname{argmin}_{\theta} \frac{1}{N} \sum_{i=1}^N (f_{\theta}(\mathbf{x}_i) - f(\mathbf{x}_i))^2 \text{ where } D = \{(\mathbf{x}_i, f(\mathbf{x}_i))\}_{i=1}^N$$

$$\text{Step 2. choose } \mathbf{x}^* = \operatorname{argmax}_{\mathbf{x}} f_{\theta^*}(\mathbf{x})$$

Problem 2. Searching an input that maximizes a proxy model can be done by gradient ascent. However, since only a small manifold of input space is valid, gradient ascent can easily fall into invalid input especially in high-dimensional space.

Prior Works – Naïve Approach



Key Challenges

Design Optimization algorithms are able to ...

1. Reason about uncertainty and out-of-distribution input values
2. Search possible input values while staying on the manifold of valid input space

Data-Driven Design Optimization (Surrogate Model-Based)

- 1. NEMO**
- 2. COMs**
- 3. RoMA**

1. Normalized Maximum Likelihood Estimation for Model-Based Optimization(NEMO)

Published as a conference paper at ICLR 2021

OFFLINE MODEL-BASED OPTIMIZATION VIA NORMALIZED MAXIMUM LIKELIHOOD ESTIMATION

Justin Fu & Sergey Levine

Department of Electrical Engineering and Computer Science
University of California, Berkeley
{justinjfu, svlevine}@eecs.berkeley.edu

1. Motivation

- **Principal Challenge of Offline Model-Based Optimization:**
 - Input space is high-dimensional, but usually with narrow manifold for valid inputs (Protein Design, Natural Image, ...)
 - Algorithms could reason about uncertainty and out-of-distribution values, since a naive optimizer can easily exploit an estimated model to return adversarial inputs.
- **New Approach:**
 - NEMO: handling uncertainty by leveraging Normalized Maximum Likelihood Estimator(NML)
 - While in the standard formulation **NML is intractable**, they propose a tractable approximation that allows us to scale the method to high-capacity neural network models.

2. Problem Setting

- **Problem Statement**

- Our objective is to find an input $\mathbf{x}$ such that

$$\text{find } \mathbf{x}^* = \operatorname{argmax} f(\mathbf{x})$$

with **only** a fixed dataset, $D = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_N, f(\mathbf{x}_N))\}$

In this study:

Problem statement. We define the offline model-based optimization (MBO) problem as follows. Assume the existence of a stochastic ground truth function $f(y|\mathbf{x})$. The MBO algorithm is given a dataset $\mathcal{D}$ of inputs $\mathbf{x}$ along with outputs y sampled from $f(y|\mathbf{x})$. Like in standard optimization problems, the goal of MBO is to find the input value that maximizes the true function:

$$\mathbf{x}^* = \operatorname{argmax}_{\mathbf{x}} \mathbb{E}_{y \sim f(y|\mathbf{x})} [y]. \quad (1)$$

3. Preliminaries

- **Conditional Normalized Maximum Likelihood(CNML)**
 - CNML distribution is the distribution closest to the MLE, assuming the test label y is chosen adversarially.
 - Formally, CNML estimator is the minmax solution of the following equation:

$$p_{NML}(y|\mathbf{x}) = \operatorname{argmin}_h \max_{y'} (\log p(y'|\mathbf{x}; \hat{\theta}_{D \cup (\mathbf{x}, y')}) - \log h(y'|\mathbf{x}))$$

where $\hat{\theta}_{D \cup (\mathbf{x}, y)} = \arg \max_{\theta} \frac{1}{N+1} \sum_{(\mathbf{x}, y) \in D \cup (\mathbf{x}, y)} \log p(y|\mathbf{x}, \theta)$ is the maximum likelihood estimate for p using the dataset $\mathcal{D}$ augmented with $(\mathbf{x}, y)$.

- Solution of the minmax problem can be expressed as:

$$p_{NML}(y|\mathbf{x}) = \frac{p(y|\mathbf{x}; \hat{\theta}_{D \cup (\mathbf{x}, y)})}{\int p(y'|\mathbf{x}; \hat{\theta}_{D \cup (\mathbf{x}, y')}) dy'}$$

- $p_{NML}(y|\mathbf{x})$ is an conservative estimate on $q(y|\mathbf{x})$

4. NEMO

- Normalized Maximum Likelihood estimator for unknown stochastic function $f(x)$

$$p_{NML}(y|\mathbf{x}) = \frac{p(y|\mathbf{x}; \hat{\theta}_{D \cup (\mathbf{x}, y)})}{\int p(y'|\mathbf{x}; \hat{\theta}_{D \cup (\mathbf{x}, y')}) dy'}$$

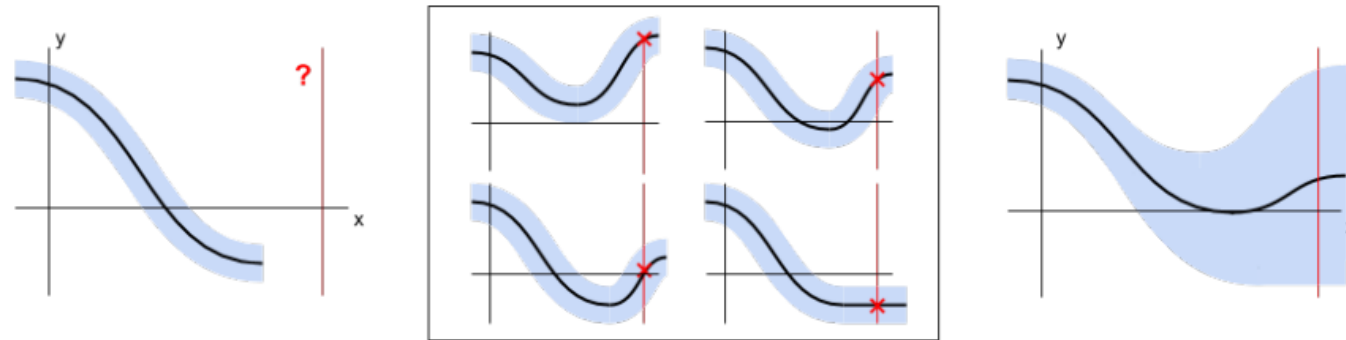


Figure 1: An illustrative example of the construction of the CNML distribution. **Left** We wish to estimate the $p(y|x)$ at some query point x , marked by the vertical red line. **Middle** We compute the MLE assuming we knew the true label y , for every possible value of y . **Right** Finally, we normalize the predictions across all MLE models to produce p_{NML} . The final prediction will likely exhibit large amounts of uncertainty on queries x far from the dataset, because it is easier for the individual MLE estimates to overfit to these outliers.

4. NEMO

- Problem: CNML Estimator is intractable!

$$p_{NML}(y|\mathbf{x}) = \frac{p(y|\mathbf{x}; \hat{\theta}_{D \cup (\mathbf{x}, y)})}{\int p(y'|\mathbf{x}; \hat{\theta}_{D \cup (\mathbf{x}, y')}) dy'}$$

- NML is intractable because the denominator requires integrating and training a MLE estimator for every possible y

Problem 1. We consider a complex neural network model for estimating the ground-truth function. Training MLE for complex neural network model for each value of y is impossible

Problem 2. Integrating estimated probability distribution for all values of y is impossible

4. NEMO

- **Solution 2: Quantization**

- If we assume that the domain of y is discrete, then the denominator is tractable
- Therefore, we quantize all outputs in the dataset by flooring each y value to the nearest bin. Then, the integral is approximated as:

$$\int_y p(y|x, \hat{\theta}_{\mathcal{D} \cup (\mathbf{x}, y)}) dy \approx B \sum_{k=1}^K p(\lfloor y_k \rfloor | x, \hat{\theta}_{\mathcal{D} \cup (\mathbf{x}, \lfloor y_k \rfloor)})$$

4. NEMO

Solution 1: Iterative Algorithm for Model-Based Optimization

- Assume that the domain of $\mathcal{Y}$ is discrete
- Straightforward: Constructing NML estimator at every iterate $\mathbf{x}_t$ (full learning) and apply gradient ascent to find $\mathbf{x}_{t+1}$
- Proposed approach: **Amortize the learning process** by **incrementally learning** the NML distribution **while optimizing the input $\mathbf{x}_t$**

Algorithm 1 NEMO: Normalized Maximum Likelihood for Model-Based Optimization

Input Model class $\{f_\theta : \theta \in \Theta\}$, Dataset $\mathcal{D} = (\mathbf{x}_{1:N}, y_{1:N})$, number of bins K , evaluation function $g(y)$, learning rates α_θ, α_x .

Initialize K models $\theta_0^{1:K}$, optimization iterate $\mathbf{x}_0$

Quantize $y_{1:N}$ into K bins, denoted as $\lfloor \mathcal{Y} \rfloor = \{\lfloor y_1 \rfloor, \dots, \lfloor y_K \rfloor\}$.

for iteration t in $1 \dots T$ **do**

for k in $1 \dots K$ **do**

 construct augmented dataset: $\mathcal{D}' \leftarrow \mathcal{D} \cup (\mathbf{x}_t, \lfloor y_k \rfloor)$.

 update model: $\theta_{t+1}^k \leftarrow \theta_t^k + \alpha_\theta \nabla_{\theta_t^k} \text{LogLikelihood}(\theta_t^k, \mathcal{D}')$

end for

 estimate CNML distribution: $\hat{p}_{\text{NML}}(y|\mathbf{x}_t) \propto p(y|\mathbf{x}_t, \theta_t^y) / \sum_k p(\lfloor y_k \rfloor | \mathbf{x}_t, \theta_t^k)$

 Update $\mathbf{x}$: $\mathbf{x}_{t+1} \leftarrow \mathbf{x}_t + \alpha_x \nabla_{\mathbf{x}} E_{y \sim \hat{p}_{\text{NML}}(y|\mathbf{x})} [g(y)]$

end for

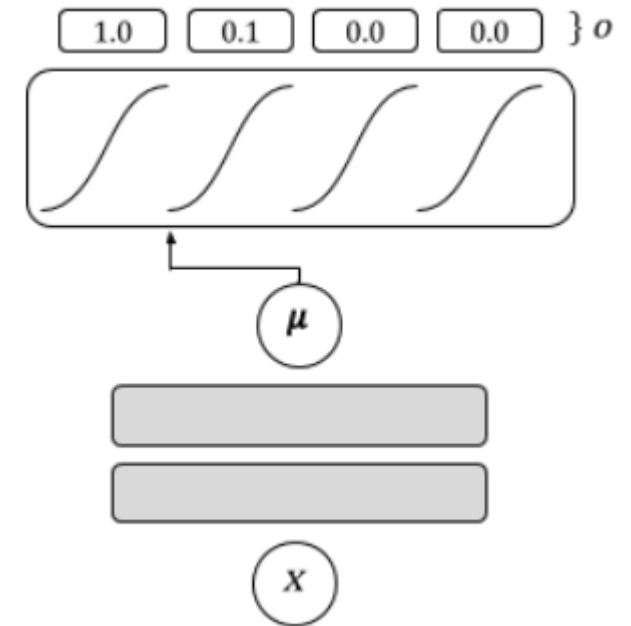
4. NEMO

• Model Architecture

- Quantization results in **flat regions in the optimization landscape**, making using gradient-based algorithms to optimize challenging
- To alleviate this issue, NEMO uses a **discretized logistic architecture**
 - Transforms input x into the mean parameter of a logistic distribution
 - Outputs one minus the CDF of a logistic distribution queried at regular intervals of $1/K$
 - Predicted y is computed as a mean over entire output vector
- Compute gradient of the model to optimize input will lead to compute gradients with respect to the logistic parameter $\mu(x)$, which is **less susceptible to flat gradients** introduced by the quantization procedure
- Furthermore, gradients with respect to $\mu(x)$ and y **share a positive angle**

Proposition 4.1 (Discretized Logistic Gradients). *Let $\mu(x)$ denote the mean of the discretized logistic architecture for input x , and $y_{\text{mean}}(x)$ denote the predicted mean. Then,*

- If $x \in \arg \max_x \mu(x)$, then $x \in \arg \max_x y_{\text{mean}}(x)$.*
- For any x , $\langle \nabla_x \mu(x), \nabla_x y_{\text{mean}}(x) \rangle \geq 0$.*



<Discretized Logistic Architecture>

5. Experiments

- **Uncertainty Estimation**

- (Quantized) NML distribution vs Bootstrapped Ensemble Method

In regions outside of the support, the NML outputs a highly uncertain estimate.

However, the ensemble still outputs highly confident estimates, even though they may be wrong



5. Experiments

- **High-Dimensional Optimization Problems**

- Evaluation on 6 tasks from Design-Bench

Task	Dimension	Type	Dataset size
Superconductor	81	Continuous	21,263
GFP	238	Categorical(20)	5,000
MoleculeActivity	1,024	Binary	4,216
HopperController	5,126	Continuous	3,200
AntMorphology	60	Continuous	25,009
DkittyMorphology	56	Continuous	25,009

5. Experiments

- **High-Dimensional Optimization Problems**

- Baselines

1. Forward: Single Forward Model
2. Ensemble: Bootstrapped Ensemble Forward Model
3. GP-BO: Bayesian Optimization with Gaussian Process and Expected Improvement
4. MINs: Model Inversion Networks
5. CbAS: Conditioned by Adaptive Sampling
6. Autofocus: Autofocused oracles

5. Experiments

- High-Dimensional Optimization Problems

- Results

Produce a batch of 128 candidates' designs, and report 50th percentile and 100th percentile
NEMO produces competitive results in line with the best performing algorithm for each task

100 th Perc.	Superconductor	GFP	MoleculeActivity
NEMO (ours)	127.0 ± 7.292	3.359 ± 0.036	6.682 ± 0.209
Ensemble	88.01 ± 10.43	2.924 ± 0.039	6.525 ± 0.1159
Forward	89.64 ± 9.201	2.894 ± 0.001	6.636 ± 0.066
MINs	80.23 ± 10.67	3.315 ± 0.033	6.508 ± 0.236
CbAS	72.17 ± 8.652	3.408 ± 0.029	6.301 ± 0.131
Autofoc.	77.07 ± 11.11	3.365 ± 0.023	6.345 ± 0.141
GP-BO	89.72 ± 0.000	2.894 ± 0.000	6.745 ± 0.000
Dataset Max	73.90	3.152	6.558
	HopperController	AntMorphology	DKittyMorphology
NEMO (ours)	2130.1 ± 506.9	393.7 ± 6.135	431.6 ± 47.79
Ensemble	1877.0 ± 704.2	-	-
Forward	1050.8 ± 284.5	399.9 ± 4.941	390.7 ± 49.24
MINs	746.1 ± 636.8	388.5 ± 9.085	352.9 ± 38.65
CbAS	547.1 ± 423.9	393.0 ± 3.750	369.1 ± 60.65
Autofoc.	443.8 ± 142.9	386.9 ± 10.58	376.3 ± 47.47
Dataset Max	1361.6	108.5	215.9

<100th percentile score and stddev>

50 th Perc.	Superconductor	GFP	MoleculeActivity
NEMO (ours)	66.41 ± 4.618	3.219 ± 0.039	5.814 ± 0.092
Ensemble	48.72 ± 2.637	2.910 ± 0.020	6.412 ± 0.123
Forward	54.06 ± 5.060	2.894 ± 0.000	6.401 ± 0.186
MINs	37.32 ± 10.50	3.135 ± 0.019	5.806 ± 0.078
CbAS	32.21 ± 7.255	3.269 ± 0.018	5.742 ± 0.123
Autofoc.	31.57 ± 7.457	3.216 ± 0.029	5.759 ± 0.158
GP-BO	72.42 ± 0.000	2.894 ± 0.000	6.373 ± 0.000
	HopperController	AntMorphology	DKittyMorphology
NEMO (ours)	390.2 ± 43.37	326.9 ± 5.229	180.8 ± 34.94
Ensemble	362.5 ± 80.09	-	-
Forward	185.0 ± 72.88	318.0 ± 12.05	255.3 ± 6.379
MINs	520.4 ± 301.5	184.8 ± 29.52	211.6 ± 13.67
CbAS	132.5 ± 23.88	267.3 ± 16.55	203.2 ± 3.580
Autofoc.	116.4 ± 18.66	176.7 ± 59.94	199.3 ± 8.909

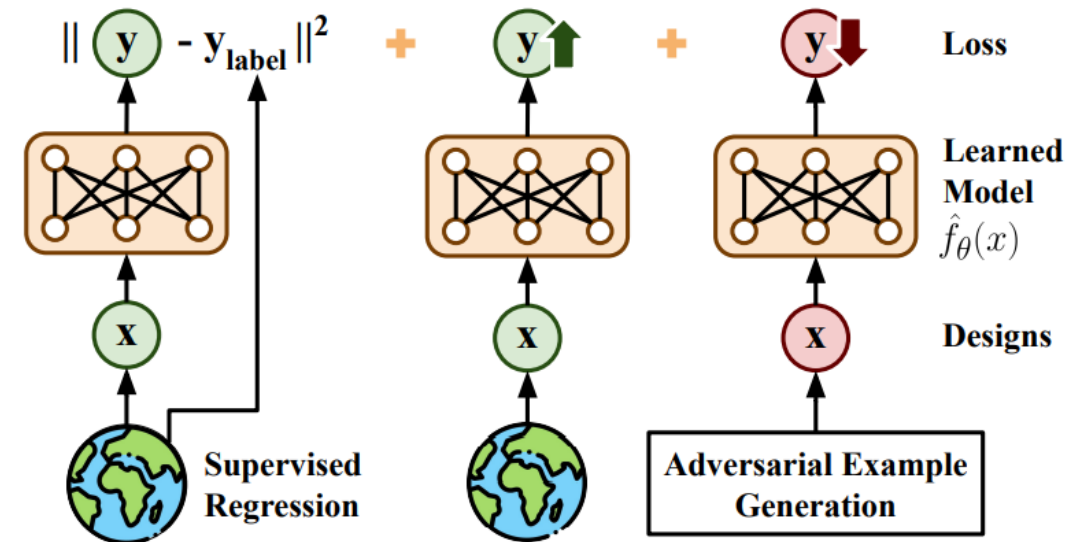
<50th percentile score and stddev>

2. Conservative Objective Models for Effective Offline Model-Based Optimization (COMs)

Conservative Objective Models for Effective Offline Model-Based Optimization

Brandon Trabucco^{*1} Aviral Kumar^{*1} Xinyang Geng¹ Sergey Levine¹

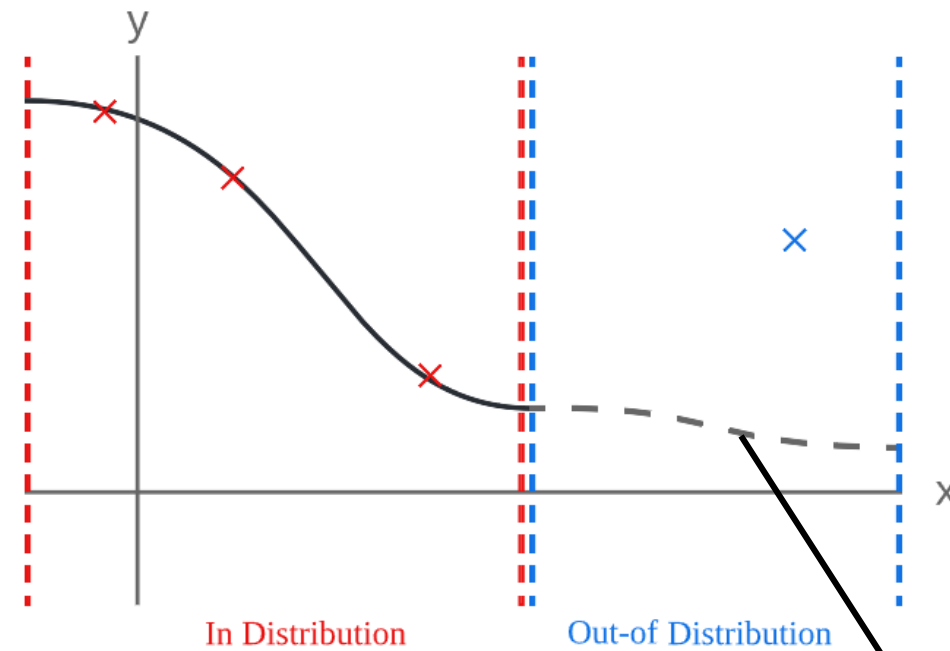
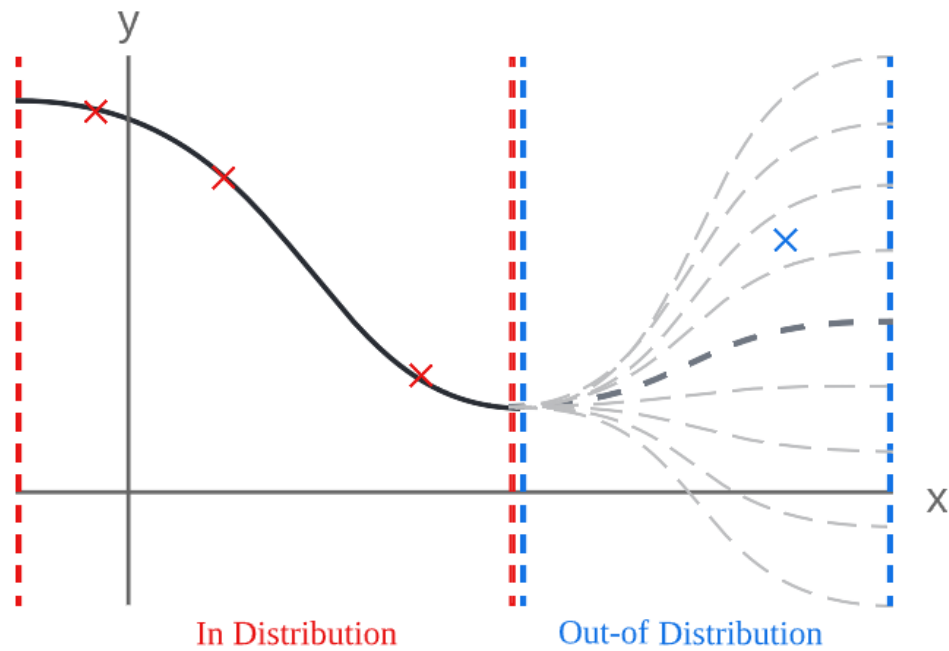
Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 2021. Copyright 2021 by the author(s).



1. Motivation

- **Principal Challenge of Offline Model-Based Optimization:**
 - Input space is high-dimensional, but usually with narrow manifold for valid inputs (Protein Design, Natural Image, ...)
 - Naïve approaches easily overestimate the objective values of out-of-distribution values, which makes an optimization process choose an invalid, low-objective value design.
- **New Approach:**
 - COMs: learn a model that **estimate the lower bound** of the ground-truth function
 - NEMO: handling uncertainty by leveraging Normalized Maximum Likelihood Estimator(NML)
 - Structurally, COMs learn a proxy model trained by conservative objective → Simple to implement, using typical optimization procedure is also possible

1. Motivation



✕: data point in offline dataset

✕: data point not in offline dataset

Learns a lower bound

2. Problem Setting

- **Problem Statement(As usual)**
 - Our objective is to find an input $\mathbf{x}$ such that

$$\text{find } \mathbf{x}^* = \operatorname{argmax}_{\mathbf{x}} f(\mathbf{x})$$

with **only** a fixed dataset, $D = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_N, f(\mathbf{x}_N))\}$

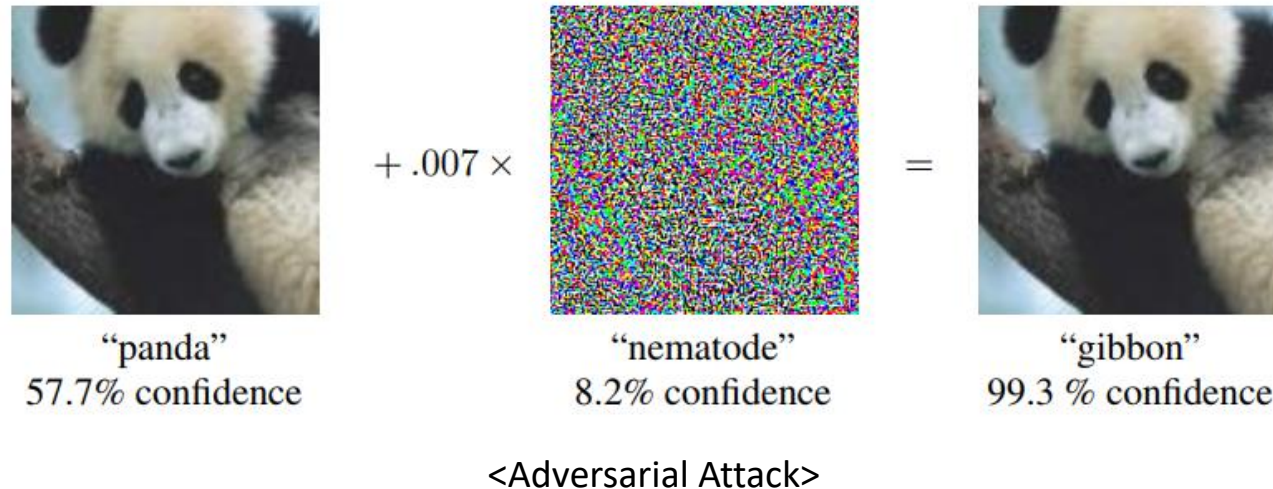
3. COMs for Offline Model-Based Optimization

- **Learning COMs**

- What we want: train the model that not overestimates output values of adversarial inputs.

Problem: How to find such adversarial inputs?

Solution: Idea from generating adversarial inputs



- Generate adversarial distribution by sampling a datapoint from the offline data and running gradient ascent:

$$\mu(\mathbf{x}) = \{\sum_{t=0} \delta_{\mathbf{x}_t} : \mathbf{x}_{t+1} = \mathbf{x}_t + \eta \cdot \nabla_{\mathbf{x}_t} \hat{f}_{\theta}(\mathbf{x}_t), \mathbf{x}_0 \sim D\}$$

3. COMs for Offline Model-Based Optimization

- **Learning COMs**

- After generating the distribution, We can simply minimize the function values under the adversarial distribution

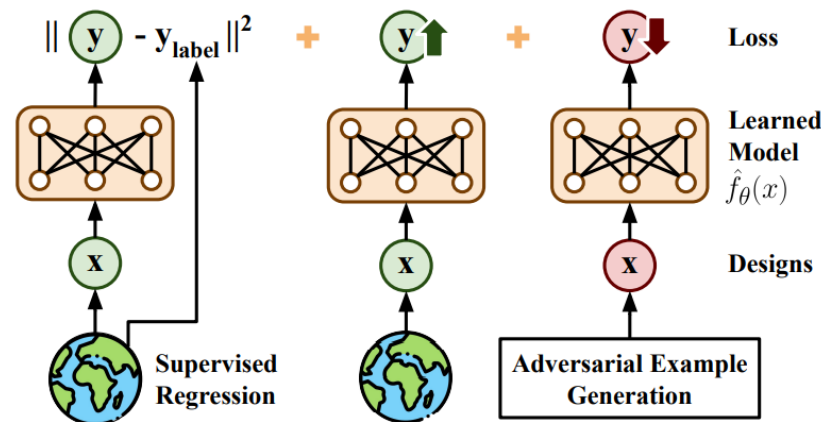
$$L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[(\hat{f}_{\theta}(\mathbf{x}) - y)^2 \right] + \alpha \cdot E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})]$$

Problem: Underestimation

Solution: Regularize by maximizing function values under the data distribution

- Final Objective

$$L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[(\hat{f}_{\theta}(\mathbf{x}) - y)^2 \right] + \alpha \cdot (E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})] - E_{\mathbf{x} \sim D} [\hat{f}_{\theta}(\mathbf{x})])$$



3. COMs for Offline Model-Based Optimization

- **Learning COMs**

- After generating the distribution, We can simply minimize the function values under the adversarial distribution

$$L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[(\hat{f}_{\theta}(\mathbf{x}) - y)^2 \right] + \alpha \cdot E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})]$$

Problem: Underestimation

Solution: Regularize by maximizing function values under the data distribution

- Final Objective

$$L(\theta) = \boxed{\frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[(\hat{f}_{\theta}(\mathbf{x}) - y)^2 \right]} + \alpha \cdot (E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})] - E_{\mathbf{x} \sim D} [\hat{f}_{\theta}(\mathbf{x})])$$

Standard MSE loss

3. COMs for Offline Model-Based Optimization

- **Learning COMs**

- After generating the distribution, We can simply minimize the function values under the adversarial distribution

$$L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[(\hat{f}_{\theta}(\mathbf{x}) - y)^2 \right] + \alpha \cdot E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})]$$

Problem: Underestimation

Solution: Regularize by maximizing function values under the data distribution

- Final Objective

$$L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[(\hat{f}_{\theta}(\mathbf{x}) - y)^2 \right] + \alpha \cdot \left(E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})] - E_{\mathbf{x} \sim D} [\hat{f}_{\theta}(\mathbf{x})] \right)$$

Prevent overestimation of out-of-distribution values

Prevent underestimation of in-distribution values

3. COMs for Offline Model-Based Optimization

- **Learning COMs(Similarity with CQL)**

- Similar to COMs , CQL also tries to learn a conservative Q-function such that the expected value of a policy under this Q-function lower-bounds its true value.

Objective of COMs:

$$\theta^* \leftarrow \operatorname{argmin}_{\theta} L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[(\hat{f}_{\theta}(\mathbf{x}) - y)^2 \right] + \alpha \cdot (E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})] - E_{\mathbf{x} \sim D} [\hat{f}_{\theta}(\mathbf{x})])$$

Objective of CQL:

$$Q^* \leftarrow \operatorname{argmin}_Q L(Q) = \frac{1}{2} E_{(s, a, s') \sim D} \left[(Q - B^{\hat{\pi}_k} \hat{Q}^k)^2 \right] + \alpha \cdot E_{s \sim D} \left(\log \sum_a \exp(Q(s, a)) - E_{a \sim \hat{\pi}_{\beta}} [Q(s, a)] \right)$$

3. COMs for Offline Model-Based Optimization

- **Theoretical Results**

- With some assumptions, conservative training lowers the bounds of the ground-truth function by Proposition 1.

Proposition 1 (Conservative training lower-bounds the true function). Assume that $\hat{f}_\theta(\mathbf{x})$ is trained with conservative training by performing gradient descent on θ with respect to the objective in Equation 7 with a learning rate η . The parameters in step k of gradient descent are denoted by θ^k , and let the corresponding conservative model be denoted as $\hat{f}_\theta^k$. Let $\mathbf{G}$, μ , $\hat{L}$, L , $\overline{\mathcal{D}}$ be defined as discussed above. Then, under assumptions listed above, $\forall \mathbf{x} \in \mathcal{D}, \mathbf{x}'' \in \mathcal{X}$, the conservative model at iteration $k + 1$ of training satisfies:

$$\begin{aligned} \hat{f}_\theta^{k+1}(\mathbf{x}'') := \max \bigg\{ & \hat{f}_\theta^{k+1}(\mathbf{x}) - \hat{L} \|\mathbf{x}'' - \mathbf{x}\|_2, \\ & \tilde{f}_\theta^{k+1}(\mathbf{x}'') - \eta\alpha \mathbb{E}_{\mathbf{x} \sim \overline{\mathcal{D}}, \mathbf{x}' \sim \mu} [\mathbf{G}_f^k(\mathbf{x}'', \mathbf{x}')] \\ & + \eta\alpha \mathbb{E}_{\mathbf{x} \sim \overline{\mathcal{D}}, \mathbf{x}' \sim \overline{\mathcal{D}}} [\mathbf{G}_f^k(\mathbf{x}'', \mathbf{x}')] \bigg\}, \end{aligned}$$

where $\tilde{f}_\theta^{k+1}(\mathbf{x}'')$ is the resulting $(k+1)$ -th iterate of $\hat{f}_\theta$ if conservative training were not used. Thus, if α is sufficiently large, the expected value of the asymptotic function, $\hat{f}_\theta := \lim_{k \rightarrow \infty} \hat{f}_\theta^k$, on inputs $\mathbf{x}_T$ found by the optimizer, lower-bounds the value of the true function $f(\mathbf{x}_T)$:

$$\mathbb{E}_{\mathbf{x}_0 \sim \mathcal{D}, \mathbf{x}_T \sim \mu(\mathbf{x}_T | \mathbf{x}_0)} [\hat{f}_\theta(\mathbf{x}_T)] \leq \mathbb{E}_{\mathbf{x}_0 \sim \mathcal{D}, \mathbf{x}_T \sim \mu(\mathbf{x}_T | \mathbf{x}_0)} [f(\mathbf{x})].$$

3. COMs for Offline Model-Based Optimization

- **Optimizing COMs**

- MINs: Choosing y to put into the inversion mapping is complicated
- NEMO: Iterative Procedure is required
- COMs: we can find an input simply by **utilizing a naïve gradient ascent optimizer!!**
Specifically, COMs starts from the input with best output value in the training dataset and performs several step of gradient ascent.

Algorithm 1 COM: Training Conservative Models

- 1: Initialize $\hat{f}_\theta$. Pick η, α and initialize dataset $\mathcal{D}$.
 - 2: **for** $i = 1$ to training_steps **do**
 - 3: Sample $(\mathbf{x}_0, y) \sim \mathcal{D}$
 - 4: Find $\mathbf{x}_T(\mathbf{x}_0)$ via gradient ascent from $\mathbf{x}_0$:
 $\mathbf{x}_{t+1} = \mathbf{x}_t + \eta \nabla_{\mathbf{x}} \hat{f}_\theta(\mathbf{x})|_{\mathbf{x}=\mathbf{x}_t}; \quad \mu(\mathbf{x}) = \sum_{\mathbf{x}_0 \in \mathcal{D}} \delta_{\mathbf{x}=\mathbf{x}_T(\mathbf{x}_0)}$
 - 5: Minimize $\mathcal{L}(\theta; \alpha)$ with respect to θ .
 $\mathcal{L}(\theta; \alpha) = \mathbb{E}_{\mathbf{x}_0 \sim \mathcal{D}} (\hat{f}_\theta(\mathbf{x}_0) - y)^2 - \alpha \mathbb{E}_{\mathbf{x}_0} [\hat{f}_\theta(\mathbf{x}_0)] + \alpha \mathbb{E}_{\mu(\mathbf{x})} [\hat{f}_\theta(\mathbf{x})]$
 $\theta \leftarrow \theta - \lambda \nabla_{\theta} \mathcal{L}(\theta; \alpha)$
 - 6: **end for**
-

Algorithm 2 COM: Finding $\mathbf{x}^*$

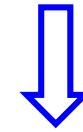
- 1: Initialize optimizer at the optimum in $\mathcal{D}$:
 $\tilde{\mathbf{x}} = \arg \max_{(\mathbf{x}, y) \in \mathcal{D}} y$
 - 2: Find $\mathbf{x}^*$ via gradient ascent from $\tilde{\mathbf{x}}$:
 $\mathbf{x}_{t+1} = \mathbf{x}_t + \eta \nabla_{\mathbf{x}} \mathcal{L}_{\text{opt}}(\mathbf{x})|_{\mathbf{x}=\mathbf{x}_t}$
 where $\mathcal{L}_{\text{opt}}(\mathbf{x}) := \hat{f}_\theta^*(\mathbf{x})$
 - 3: Return the solution $\mathbf{x}^* = \mathbf{x}_T$.
-

3. COMs for Offline Model-Based Optimization

- **Some Practical Considerations**

- **Choosing α is difficult** $\rightarrow$ modify training objective to constrained optimization problem
- Even though we have a new hyperparameter τ , **it is easy to handle since changing τ does not harm loss value.**

$$\theta^* \leftarrow \operatorname{argmin}_{\theta} L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[(\hat{f}_{\theta}(\mathbf{x}) - y)^2 \right] + \alpha \cdot (E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})] - E_{\mathbf{x} \sim D} [\hat{f}_{\theta}(\mathbf{x})])$$



$$\theta^* \leftarrow \operatorname{argmin}_{\theta} L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[(\hat{f}_{\theta}(\mathbf{x}) - y)^2 \right] \quad \text{s. t. } E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})] - E_{\mathbf{x} \sim D} [\hat{f}_{\theta}(\mathbf{x})] \leq \tau$$

- Converting training objective as a constrained optimization problem is prevalent(ex. TRPO)

4. Experiments

- **Benchmark Tests**

- Similar with NEMO, evaluation is performed on 6 tasks from Design-Bench

Method	Discrete domain		Continuous domain				Avg. [†]
	GFP	Molecule	Supercond.	Hopper	Ant	Dkitty	
Dataset Max	3.152	6.558	73.90	1361.6	108.5	215.9	1.000
CbAS [5]	3.408±0.029	6.301±0.131	72.17±8.652	547.1±423.9	393.0±3.750	396.1±60.65	1.324
Autofocus [8]	3.365±0.023	6.345±0.141	77.07±11.11	443.8±142.9	386.9±10.58	376.3±47.47	1.286
NEMO [11]	3.359±0.036	6.682±0.209	127.0±7.292	2130.1±506.9	393.7±6.135	431.6±47.79	1.687
MINs [25]	3.315±0.029	6.508±0.236	80.23±10.67	746.1±636.8	388.5±9.085	352.9±38.65	1.304
COMs [52]	3.305±0.024	6.876±0.128	110.0±6.804	2395.7±561.7	378.8±10.01	341.4±28.47	1.589
Grad. Ascent [53]	2.894±0.001	6.636±0.066	89.64±9.201	1050.8±284.5	399.9±4.941	390.7±49.24	1.237

4. Experiments

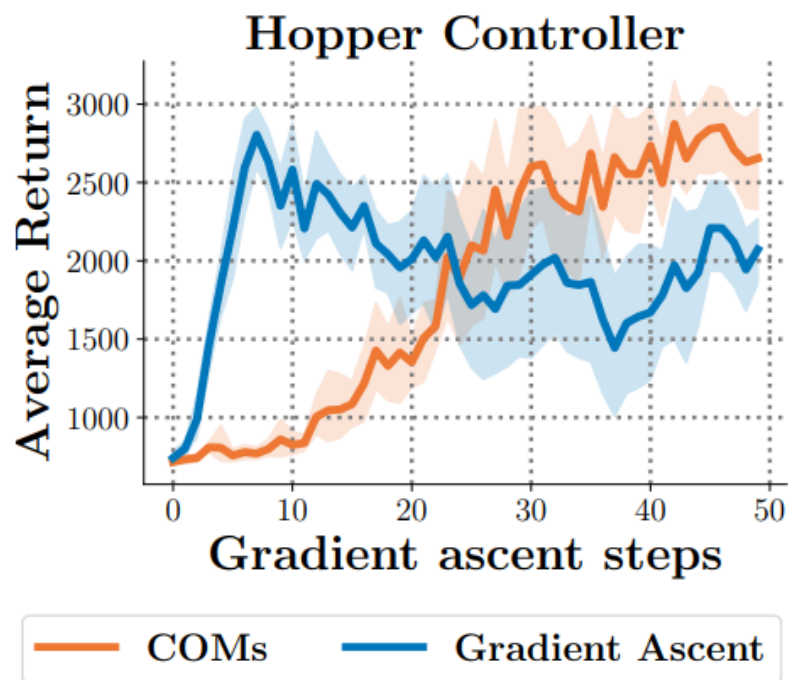
- **Ablation Studies**

- COMs vs Naive Gradient Ascent

$$\theta^* \leftarrow \operatorname{argmin}_{\theta} L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[\left(\hat{f}_{\theta}(\mathbf{x}) - y \right)^2 \right]$$

$$\text{s. t. } E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})] - E_{\mathbf{x} \sim D} [\hat{f}_{\theta}(\mathbf{x})] \leq \tau$$

evaluate the effect of this term



4. Experiments

- **Ablation Studies**

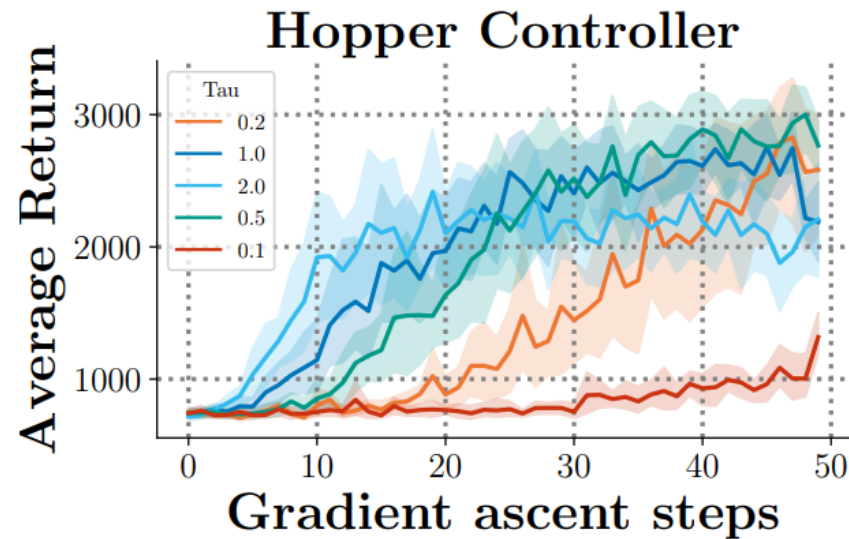
- Sensitivity Test for τ

$$\theta^* \leftarrow \operatorname{argmin}_{\theta} L(\theta) = \frac{1}{2} E_{(\mathbf{x}, y) \sim D} \left[\left(\hat{f}_{\theta}(\mathbf{x}) - y \right)^2 \right]$$

$$\text{s. t. } E_{\mathbf{x} \sim \mu(\mathbf{x})} [\hat{f}_{\theta}(\mathbf{x})] - E_{\mathbf{x} \sim D} [\hat{f}_{\theta}(\mathbf{x})] \leq \tau$$



Important hyperparameter of the model,
Should be robust



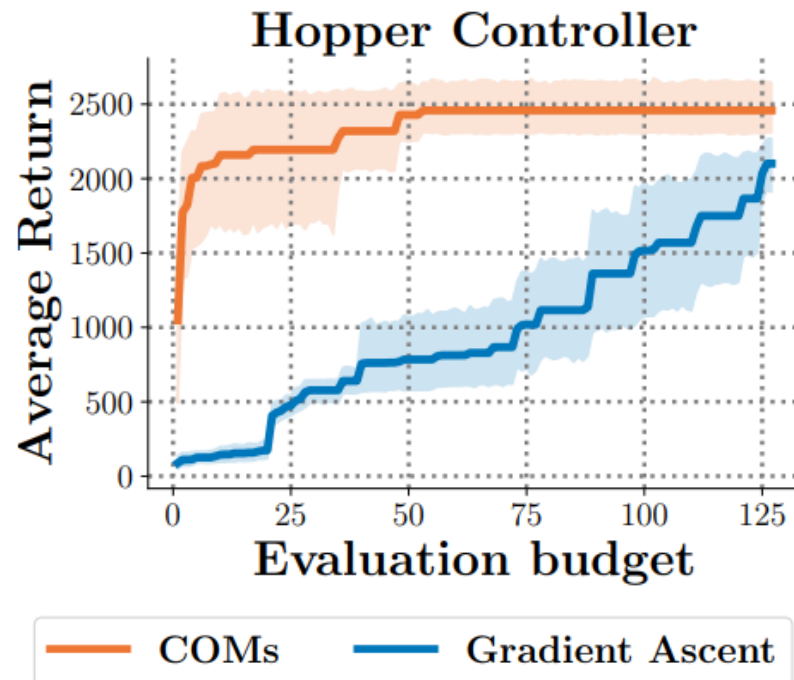
4. Experiments

- **Ablation Studies**

- Reduce the number of candidates

Evaluation of Offline Model-Based Optimization is performed by selecting N candidates from the model and show 50th and 100th percentile results.

Researchers show that COMs are resilient to N



3. Robust Model Adaptation for Offline Model-Based Optimization(RoMA)

RoMA: Robust Model Adaptation for Offline Model-based Optimization

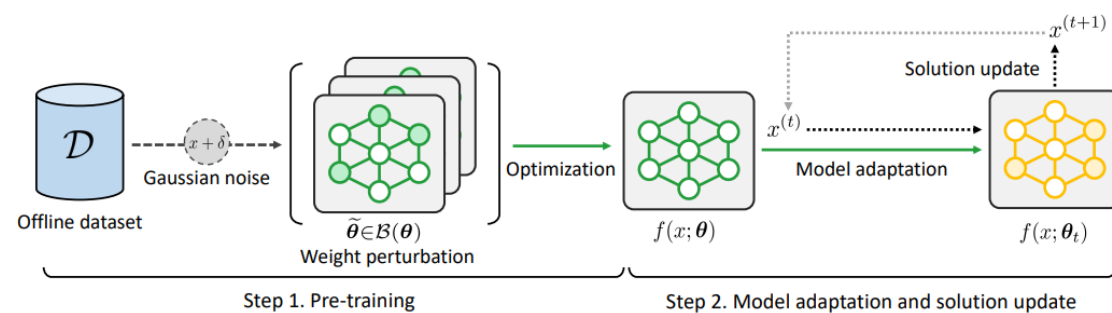
Sihyun Yu¹ Sungsoo Ahn² Le Song^{2,3} Jinwoo Shin¹

¹Korea Advanced Institute of Science and Technology (KAIST)

²Mohamed bin Zayed University of Artificial Intelligence (MBZUAI) ³BioMap

{sihyun.yu, jinwoos}@kaist.ac.kr

{Peter.Ahn, Le.Song}@mbzuai.ac.ae



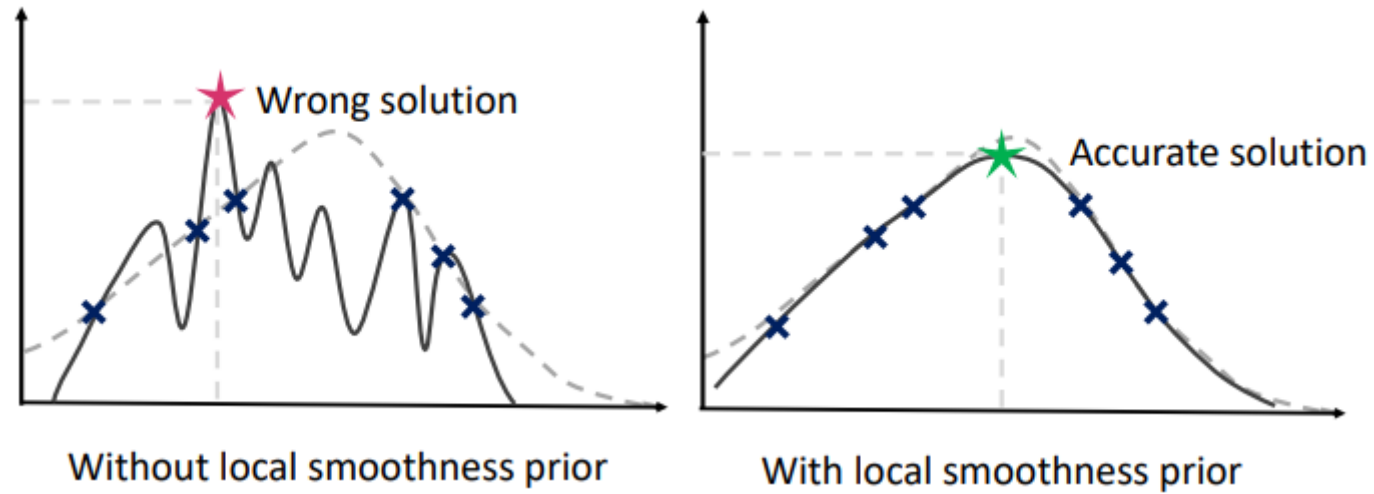
35th Conference on Neural Information Processing Systems (NeurIPS 2021).

1. Motivation

- **Principal Challenge of Offline Model-Based Optimization:**
 - Input space is high-dimensional, but usually with a narrow manifold for valid inputs (Protein Design, Natural Image, ...)
 - Naïve approaches easily overestimate the objective values of out-of-distribution values, which makes an optimization process choose invalid, low-objective value design **because of the non-smooth nature of DNN**
- **New Approach:**
 - NEMO: handling uncertainty by leveraging Normalized Maximum Likelihood Estimator(NML)
 - COMs: learn a model that estimate the lower bound of the ground-truth function
 - RoMA: utilizes a **local smoothness prior at adversarial samples**

1. Motivation

- Illustration of Local Smoothness Prior



2. Problem Setting

- **Problem Statement(As usual)**
 - Our objective is to find an input $\mathbf{x}$ such that

$$\text{find } \mathbf{x}^* = \operatorname{argmax}_{\mathbf{x}} f(\mathbf{x})$$

with **only** a fixed dataset, $D = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_N, f(\mathbf{x}_N))\}$

2. Problem Setting

- **Overall Procedure**

To be specific, our RoMA is a two-stage framework, described as follows.

Step 1. Train the proxy model on the dataset $\mathcal{D}$ to approximate the true objective function with Gaussian smoothing of inputs under worst-case weight perturbations.

Step 2. For time-steps $t = 0, \dots, T - 1$, update the solution $x^{(t)}$ after adapting the output of the proxy model to be locally smooth at the solution $x^{(t)}$.

We provide the detailed description of our framework in Algorithm 1.

3. RoMA: Robust Model Adaptation

- **Training a proxy model**

- Training a DNN-based proxy model without regularization is brittle
- RoMA focused on regularizing the training with **Gaussian smoothing of inputs** under **worst-case weight perturbations** by minimizing the following loss:

$$L(\theta) = \max_{\tilde{\theta} \in B(\theta)} \left[E_{(x, y) \sim D, \delta \sim N(0, \sigma)} \left[(f(x + \delta; \tilde{\theta}) - y)^2 \right] \right]$$

$$\text{where } B(\theta) = \left\{ \tilde{\theta} : \|\theta_l - \tilde{\theta}_l\|_F \leq \epsilon \cdot \|\theta_l\|_F \ \forall l \in \{1, \dots, L\} \right\}$$

- Regularizing with worst-case weight perturbations is proposed in *Adversarial Weight Perturbation Helps Robust Generalization, NIPS 2020*

We propose **Adversarial Weight Perturbation (AWP)** to explicitly regularize the weight loss landscape of adversarial training, forming a double-perturbation mechanism that injects the worst-case input and weight perturbations.

3. RoMA: Robust Model Adaptation

- **Training a proxy model**

- Training a DNN-based proxy model without regularization is brittle
- RoMA focused on regularizing the training with **Gaussian smoothing of inputs** under **worst-case weight perturbations** by minimizing the following loss:

$$L(\boldsymbol{\theta}) = \max_{\tilde{\boldsymbol{\theta}} \in B(\boldsymbol{\theta})} \left[E_{(x, y) \sim D, \delta \sim N(0, \sigma)} \left[(f(x + \delta; \tilde{\boldsymbol{\theta}}) - y)^2 \right] \right]$$

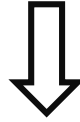
$$\text{where } B(\boldsymbol{\theta}) = \left\{ \tilde{\boldsymbol{\theta}} : \|\theta_l - \tilde{\theta}_l\|_F \leq \epsilon \cdot \|\theta_l\|_F \ \forall l \in \{1, \dots, L\} \right\}$$

- Inner maximization problem is solved using **projected gradient descent**

3. RoMA: Robust Model Adaptation

- **Iterative proxy model adaptation**

- With the pre-trained model, one may apply naive gradient ascent to optimize the model.
- However, since **smoothing to the proxy model is applied only for the training set**, gradient-based updates may be erroneous for out-of-distribution values.



- To circumvent this issue, we **adjust the proxy model to be smooth at the intermediate solution** while applying gradient ascent.
- Formally, the model parameter is updated by the following objective:

$$\theta_t = \arg \min_{\tilde{\theta} \in \mathcal{B}(\theta)} \left[\left\| \nabla_x f(x; \tilde{\theta}) \right\|_2 \Big|_{x=x^{(t)}} + \alpha \left(f(x^{(t)}; \tilde{\theta}) - f(x^{(t)}; \theta_{t-1}) \right)^2 \right]$$

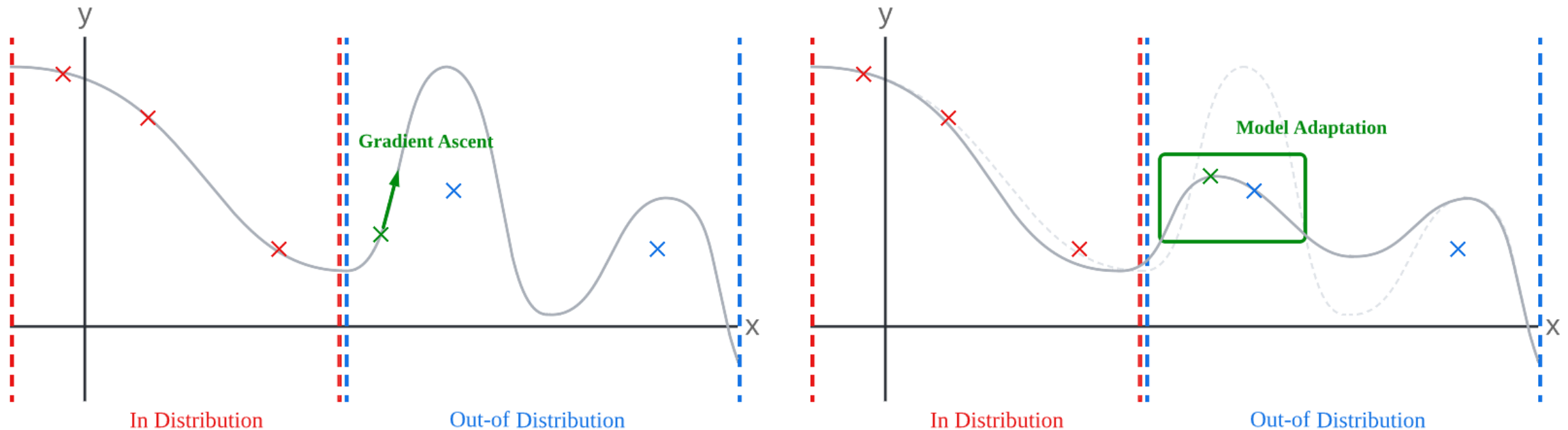
for local smoothness

Prevent model parameter going to far

3. RoMA: Robust Model Adaptation

- **Iterative proxy model adaptation**

- To circumvent this issue, we **adjust the proxy model to be smooth at the intermediate solution** while applying gradient ascent.



✕: data point in offline dataset ✕: data point not in offline dataset ✕: intermediate solution updated by gradient ascent

4. Experiments

- **Benchmark Tests**

- Similar with NEMO, evaluation is performed on 6 tasks from Design-Bench

Table 1: Comparison of 100th percentile scores for each task. We mark the scores within one standard deviation from the highest average score to be bold.

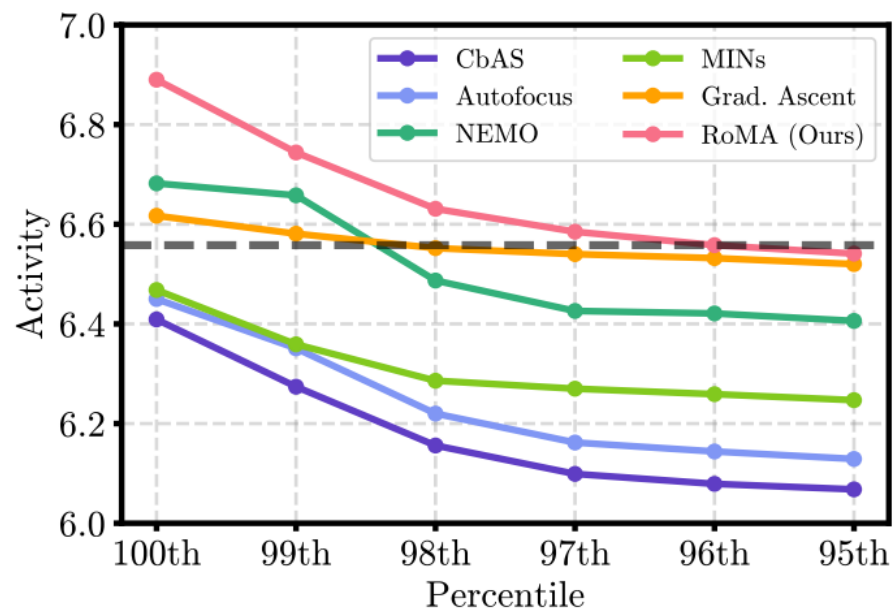
Method	Discrete domain		Continuous domain				Avg. [†]
	GFP	Molecule	Supercond.	Hopper	Ant	Dkitty	
Dataset Max	3.152	6.558	73.90	1361.6	108.5	215.9	1.000
CbAS [5]	3.408±0.029	6.301±0.131	72.17±8.652	547.1±423.9	393.0±3.750	396.1±60.65	1.324
Autofocus [8]	3.365±0.023	6.345±0.141	77.07±11.11	443.8±142.9	386.9±10.58	376.3±47.47	1.286
NEMO [11]	3.359±0.036	6.682±0.209	127.0±7.292	2130.1±506.9	393.7±6.135	431.6±47.79	1.687
MINs [25]	3.315±0.029	6.508±0.236	80.23±10.67	746.1±636.8	388.5±9.085	352.9±38.65	1.304
COMs [52]	3.305±0.024	6.876±0.128	110.0±6.804	2395.7±561.7	378.8±10.01	341.4±28.47	1.589
Grad. Ascent [53]	2.894±0.001	6.636±0.066	89.64±9.201	1050.8±284.5	399.9±4.941	390.7±49.24	1.237
RoMA (Ours)	3.357±0.024	6.890±0.122	103.9±5.487	2466.5±359.2	468.5±12.68	384.3±51.68	1.705

4. Experiments

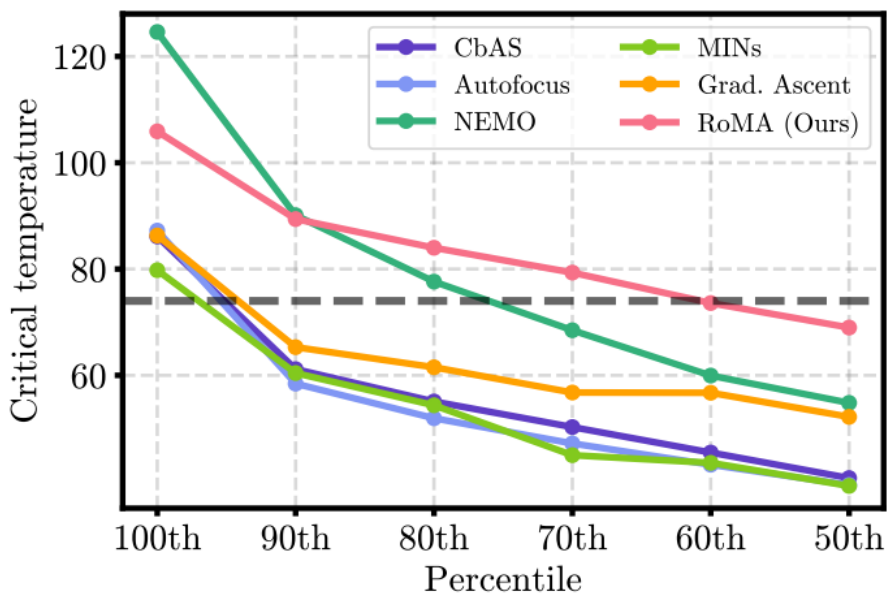
- **Ablation Studies**

- Ratio of high-scoring solutions

Because of local smoothing prior, we can expect that RoMA relatively optimizes solution candidates to other previous approaches with the higher ratio.



(a) Molecule

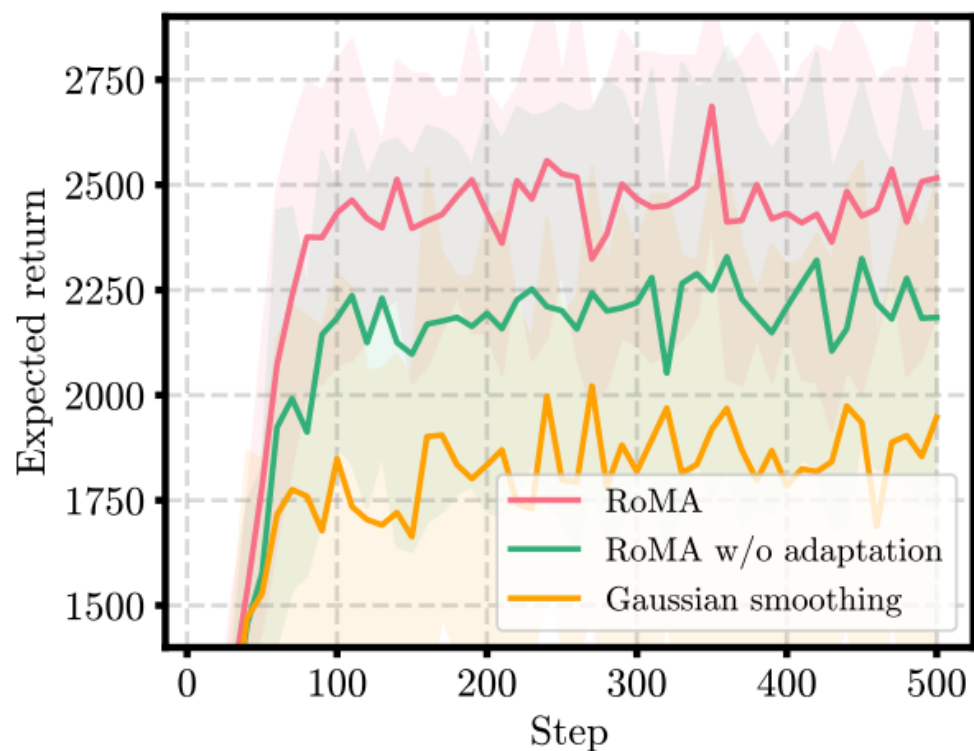


(b) Superconductor

4. Experiments

- **Ablation Studies**

- Effect of each component
 - (a) RoMA w/o model adaptation
 - (b) Gaussian Smoothing: RoMA w/o model adaptation & weight perturbations



Data-Driven Design Optimization (Surrogate Model-Based)

Applications

1. Data-Driven Offline Optimization for Architecting Hardware Accelerators (ICLR 2022)

- Develop a data-driven, offline optimization for designing hardware accelerators
- Train a conservative surrogate model similar to COMs

$$\theta^* = \operatorname{argmin}_{\theta} \mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}_i, y_i \sim \mathcal{D}} [(f_{\theta}(\mathbf{x}_i) - y_i)^2] - \alpha \mathbb{E}_{\mathbf{x}_i^- \sim \operatorname{Opt}(f_{\theta})} [f_{\theta}(\mathbf{x}_i^-)]$$

- To satisfy various constraints, **incorporate infeasible dataset as additional negative samples** for regularization and maximize the prediction at these points

$$\theta^* = \operatorname{argmin}_{\theta} \mathcal{L}(\theta) - \beta \mathbb{E}_{\mathbf{x}'_i \sim \mathcal{D}_{\text{infeasible}}} [f_{\theta}(\mathbf{x}'_i)]$$

- To be effective for new unseen application domains, **extend the training procedure to incorporate context vectors** and train the surrogate model in expectation over all contexts

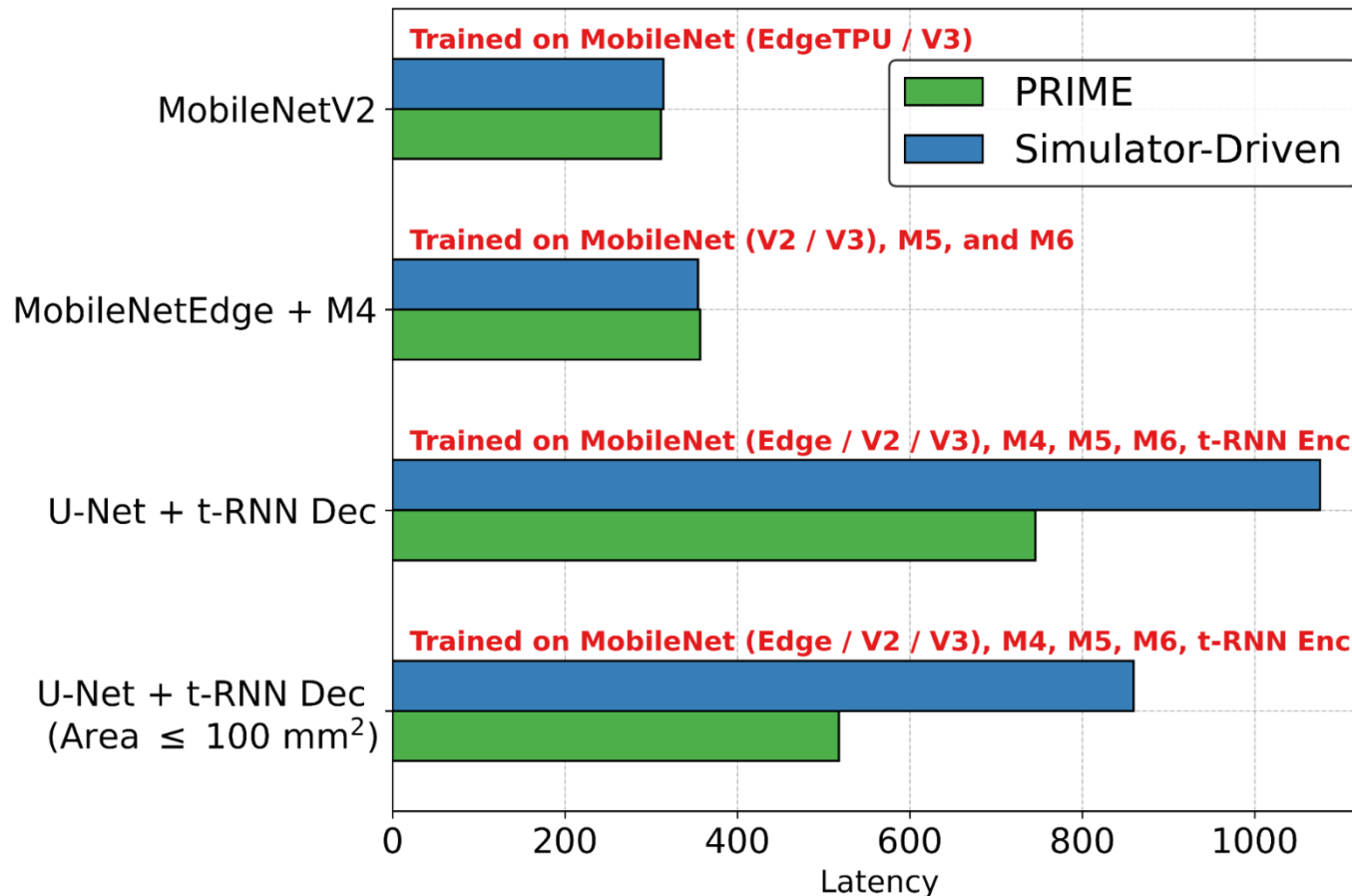
$$\theta^* = \operatorname{argmin}_{\theta} \mathbb{E}_{k \sim [K]} [\mathcal{L}_k(\theta)],$$

$$\mathcal{L}_k(\theta) = \mathbb{E}_{\mathbf{x}_i, y_i \sim \mathcal{D}} [(f_{\theta}(\mathbf{x}_i, \mathbf{c}_k) - y_i)^2] - \alpha \mathbb{E}_{\mathbf{x}_i^- \sim \operatorname{Opt}(f_{\theta})} [f_{\theta}(\mathbf{x}_i^-, \mathbf{c}_k)]$$

Cutting-Edge Papers

1. Data-Driven Offline Optimization for Architecting Hardware Accelerators (ICLR 2022)

- Develop a data-driven, offline optimization for designing hardware accelerators
- Demonstration Results (Lower Latency is Better)



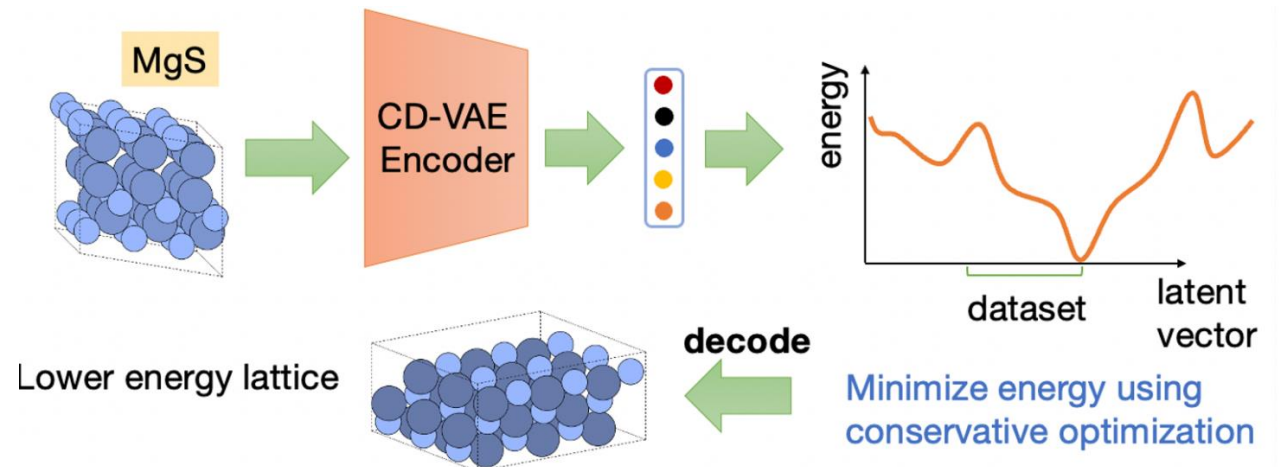
2. Latent Conservative Objective Models for Data-Driven Crystal Structure Prediction (ICLR 2022)

- Develop a data-driven, offline optimization for discovering the lowest energy-stable crystal structure for a given chemical formula.
- As the chemical space is non-Euclidean space, we utilize an auto-encoder (CD-VAE) to convert a crystal structure into a vector-based search space and then optimize a conservative surrogate model of the crystal energy

- Loss function can be derived as:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}, c \sim \mathcal{D}} \left[\left(E_{\theta}(\phi(\mathbf{x}, c), c) - E(\mathbf{x}, c) \right)^2 \right] - \alpha \mathbb{E}_{\mathbf{x}^- \sim \operatorname{Opt}(E_{\theta})} [E_{\theta}(\phi(\mathbf{x}^-, c), c)]$$

$\phi(x, c)$: pre-trained encoder



- [illegible]

Questions?

?

References

- [1] Trabucco, Brandon, et al. "Design-bench: Benchmarks for data-driven offline model-based optimization." *arXiv preprint arXiv:2202.08450* (2022).
- [2] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams and N. de Freitas, "Taking the Human Out of the Loop: A Review of Bayesian Optimization," in *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148-175, Jan. 2016, doi: 10.1109/JPROC.2015.2494218.
- [3] Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. "Explaining and harnessing adversarial examples." *arXiv preprint arXiv:1412.6572* (2014).